



FACULTAD DE
INTELIGENCIA ARTIFICIAL
E INGENIERÍAS

MATERIAL DE PRÁCTICA · PYTHON

Técnicas de programación

Taller y Evaluación · Unidad 1

De los ciclos a las estructuras lineales. Cada problema pide hacer explícitos los datos, las reglas y la estructura elegida.

01

Repaso dirigido

02

Estructuras básicas

03

Objetos

04

Pilas y colas

05

Integración

20 ejercicios

numeración continua

5 bloques

progresión temática

Evaluación

parcial en papel

Taller y Evaluación — Unidad 1

Taller integrado

Técnicas de Programación · De los ciclos a las estructuras lineales

Este taller recorre todo lo que hemos visto: ciclos, listas, tuplas, diccionarios, funciones, objetos y estructuras lineales. Es la preparación para el parcial.

20 ejercicios	numeración continua
5 bloques	progresión temática
Modalidad	diseñar · implementar · verificar

Ruta del taller

BLOQUE	EJERCICIOS
1 · Repaso dirigido SIN COMPUTADOR	1 Conceptos · 2 Trazas de estructuras · 3 ¿Qué imprime?
2 · Lógica y estructuras básicas	4 Consolidación de notas · 5 Inventario de la cafetería · 6 Control de asistencia · 7 Bitácora de sensores · 8 Agenda de la sala de cómputo
3 · Objetos	9 Producto con validación · 10 Pedido como colección · 11 Pedido con descuento · 12 Medios de pago · 13 Esquema de evaluación
4 · Pilas y colas	14 Navegador con historial · 15 Editor con deshacer y rehacer · 16 Fila de la cafetería · 17 Notificaciones del celular · 18 Turnos rotativos del laboratorio
5 · Integración	19 Sala de urgencias · 20 Despacho de domicilios

Antes de programar

Propósito

Construir las soluciones **a mano**, justificar las estructuras elegidas y comprobar que se respetan las reglas de cada problema. El taller da los enunciados y las orientaciones; la implementación es tuya.

Reglas generales

- **No uses librerías externas.** De la biblioteca estándar solo se admite `collections.deque` cuando el ejercicio lo diga.

- Usa listas, tuplas, diccionarios, ciclos y condicionales donde corresponda. **Evita** `pop(0)` sobre una lista para simular una cola: obliga a correr de lugar todos los demás elementos.
- En los ejercicios del bloque 4 usa tus clases `Pila`, `Cola` y `ColaCircular`, o las del cuaderno *Estructuras lineales*. **No** resuelvas con listas sueltas.
- **Valida las entradas.** Deja explícitas tus convenciones y no escondas errores devolviendo resultados arbitrarios. Los ejemplos de cada ejercicio son independientes entre sí.

Cómo documentar cada solución

1. **Datos y reglas:** entradas, salidas, supuestos y validaciones.
2. **Diseño:** qué funciones o clases vas a escribir y por qué esa estructura.
3. **Implementación:** código legible, con comentarios donde haga falta.
4. **Verificación:** un caso normal, uno límite y uno inválido, con el resultado esperado.

IMPORTANTE

El **bloque 1 se resuelve sin computador**: papel y lápiz. En el parcial no vas a tener el intérprete para salir de dudas. Escribe primero tu respuesta y **después** verifícala ejecutando. Si tuviste que correr el código para saber qué pasaba, ese es justamente el tema que hay que repasar.

Dónde están las clases

Pega aquí las clases `Pila`, `Cola` y `ColaCircular` antes de empezar el bloque 4.

1 · Conceptos

Responde en **dos o tres líneas** cada una. Con tus palabras, no copiando definiciones.

Ciclos

1. ¿Cuándo usarías `for` y cuándo `while`? Da un caso incómodo de resolver con `for`.
2. Diferencia entre `break`, `continue` y `pass`. ¿Cuál **no** altera el ciclo?
3. ¿Qué imprime `range(2, 9, 3)` y por qué no aparece el 9?

Listas, tuplas y diccionarios

4. Una lista y una tupla guardan lo mismo. ¿Por qué elegirías una tupla?
5. Una tupla solo tiene `count()` e `index()`. ¿Por qué no tiene `append` ni `sort`?
6. Diferencia entre `d["telefono"]` y `d.get("telefono")` cuando la clave **no existe**.
7. Tras `b = a` sobre una lista, haces `b.append(5)`. ¿Qué le pasa a `a`? ¿Por qué?

Objetos

8. ¿Qué es `self` y por qué hay que escribirlo?
9. En Python no existe `private`. ¿Qué se usa en su lugar y qué gana quien lee tu código?
10. Escribiste `__len__` pero nunca un método `size`. Aun así `len(obj)` funciona. ¿Por qué?
11. Dos clases sin superclase común funcionan con la misma función. ¿Cómo se llama eso?

Estructuras lineales

12. Una pila guarda lo mismo que una lista pero con menos operaciones. ¿Qué se gana restringiendo?
13. ¿Para qué sirve una cola **circular** si ya existe la normal? Da un caso donde sea la correcta.
14. En una cola circular de capacidad 4, el arreglo es `['E', 'F', 'C', 'D']` y `frente = 2`. ¿En qué orden van a salir los elementos?

2 · Trazas de estructuras

Sin ejecutar nada. Escribe el resultado y solo después comprueba.

2.1 · PILA

Partes de una pila **vacía**:

```
apilar("A") → apilar("B") → apilar("C") → desapilar() → apilar("D") → apilar("E") →
desapilar() → desapilar() → apilar("F")
```

Responde: a) qué retornó cada uno de los tres `desapilar()` · b) cuál es la **cima** al final · c) cuántos elementos quedan · d) la pila final **de la base a la cima**.

2.2 · COLA

Partes de una cola **vacía**:

```
encolar(10) → encolar(20) → encolar(30) → desencolar() → encolar(40) → desencolar()
→ encolar(50)
```

Responde: a) qué retornó cada `desencolar()` · b) **frente** y **final** al terminar · c) la cola del frente hacia atrás.

2.3 · PILA Y COLA A LA VEZ

Tienes **P** (pila) y **C** (cola), las dos vacías:

1. En **las dos**, mete 1 , luego 2 , luego 3
2. Saca **uno** de **P** y mételo en **C**
3. Saca **uno** de **C** y mételo en **P**
4. Saca **uno** de **P**

Responde: qué quedó en **P** , qué quedó en **C** , y qué valor salió en el paso 4.

2.4 · COLA CIRCULAR

Cola circular de **capacidad 4**, vacía:

```
encolar("L") → encolar("M") → encolar("N") → encolar("O") → encolar("P") →
desencolar() → desencolar() → encolar("P") → encolar("Q")
```

Responde: a) qué pasó exactamente en el quinto paso · b) dibuja el **arreglo físico** final con sus índices · c) en qué posición física quedó la **"Q"** · d) el **orden lógico** final · e) cuánto vale **frente** .

3 · ¿Qué imprime?

Para cada celda, **escribe la salida antes de ejecutarla**.

```
# 3.1
datos = [4, 7, 2, 9, 2, 5]
total = 0
for i, n in enumerate(datos):
    if n == 2:
        continue
    if n > 8:
        break
    total += n
print(total, i)
```

```
# 3.2
inventario = {"lapiz": 3, "cuaderno": 1}
inventario["borrador"] = inventario.get("borrador", 0) + 5
inventario["lapiz"] += 2
del inventario["cuaderno"]

print(len(inventario))
print(sorted(inventario))
print(inventario.get("cuaderno", "agotado"))
```

```
# 3.3
class Carrito:
    def __init__(self, items=[]):
        self.items = items
    def agregar(self, x):
        self.items.append(x)

a = Carrito()
b = Carrito()
a.agregar("pan")
print(a.items, b.items)
```

```
# 3.4
class Empleado:
    def __init__(self, nombre):
        self.nombre = nombre
    def saludo(self):
        return f"Soy {self.nombre}"

class Jefe(Empleado):
    def __init__(self, nombre, area):
        super().__init__(nombre)
        self.area = area
    def saludo(self):
        return super().saludo() + f", jefe de {self.area}"

print(Empleado("Ana").saludo())
print(Jefe("Luis", "Sistemas").saludo())
print(isinstance(Jefe("Luis", "Sistemas"), Empleado))
```

```
# 3.5
class Caja:
    def __init__(self, cosas):
        self.__cosas = list(cosas)
    def __len__(self):
        return len(self.__cosas)
    def __contains__(self, x):
        return x in self.__cosas
    def __iter__(self):
        return iter(tuple(self.__cosas))

c = Caja(["tuerca", "tornillo", "clavo"])
print(len(c))
print("clavo" in c)
print([x.upper() for x in c])
print(bool(Caja([])))
```

4 · Consolidación de notas

El programa necesita un resumen de los promedios del curso y cuántos estudiantes hay en cada estado.

Entrada. Lista de diccionarios: {"estudiante": "Ana", "notas": [4.0, 3.2, 4.5]}

Salida. resumen : lista de tuplas (nombre, promedio, estado) · conteo : cuántos por estado.

QUÉ DEBE HACER

1. Calcula el promedio de cada estudiante **acumulando con un ciclo**. No uses `sum()`.
2. Clasifica: **Excelente** si el promedio es al menos 4.5; **Aprobado** si está entre 3.0 inclusive y 4.5 exclusive; **Reprobado** si es menor que 3.0.
3. Construye `resumen` y ordénalo de mayor a menor promedio.
4. Cuenta los estudiantes de cada estado, **incluso los estados que queden en cero**.
5. Decide qué hacer con un estudiante **sin notas**: rechazarlo o reportarlo, pero no meterlo arbitrariamente en un estado.

EJEMPLO DE SALIDA

```
resumen = [("Luis", 4.6, "Excelente"), ("Ana", 3.9, "Aprobado")]
conteo = {"Excelente": 1, "Aprobado": 1, "Reprobado": 0}
```

RESTRICCIONES

Ciclos y condicionales. Nada de funciones estadísticas integradas.

PRUEBAS

Promedios exactamente 3.0 y 4.5, estudiantes con distinta cantidad de notas, lista vacía y un estudiante sin notas.

5 · Inventario de la cafetería

La cafetería registra cada venta por separado y necesita consolidar el día.

Entrada. Lista de tuplas (producto, cantidad, precio_unitario) .

Salida. resumen, agotados y ranking .

QUÉ DEBE HACER

1. Recorre las ventas y **agrupa por producto** en un diccionario, acumulando unidades e ingreso.
2. Guarda en `resumen[producto]` la tupla `(unidades_totales, ingreso_total)` .
3. Si un producto superó **30 unidades**, agrégalo a `agotados` con el motivo `"reabastecer"` .
4. Construye `ranking` con tuplas `(producto, ingreso_total)` , de mayor a menor ingreso.
5. Rechaza registros con cantidad cero o negativa, o precio no positivo, y repórtalos aparte.

EJEMPLO DE SALIDA

```
resumen = {"Empanada": (45, 90000.0), "Jugo": (12, 36000.0)}
agotados = {"Empanada": "reabastecer"}
ranking = [("Empanada", 90000.0), ("Jugo", 36000.0)]
```

RESTRICCIONES

Al menos un `for` , condicionales, y listas, tuplas o diccionarios. Sin librerías externas.

PRUEBAS

Lista vacía, productos repetidos, exactamente 30 unidades y registros inválidos.

6 · Control de asistencia

Se tiene el registro de entrada de cada sesión y hay que detectar quién está en riesgo por fallas.

Entrada. Diccionario {"sesion_1": ["Ana", "Luis"], "sesion_2": ["Ana"]} y el total de sesiones.

Salida. asistencias por estudiante, porcentajes y en_riesgo .

QUÉ DEBE HACER

1. Recorre las sesiones y cuenta a cuántas asistió cada estudiante.
2. Calcula el porcentaje de asistencia de cada uno sobre el total de sesiones.
3. Marca en `en_riesgo` a quien tenga **menos del 85 %**.
4. Un estudiante que aparece en una sesión y no en otra **no** debe desaparecer del resumen: debe quedar con las asistencias que sí tuvo.
5. Si un nombre aparece **dos veces en la misma sesión**, cuéntalo una sola vez y repórtalo.

EJEMPLO DE SALIDA

```
asistencias = {"Ana": 2, "Luis": 1}
porcentajes = {"Ana": 100.0, "Luis": 50.0}
en_riesgo    = ["Luis"]
```

RESTRICCIONES

Prohibido usar `set` para quitar repetidos. Resuélvelo con diccionarios, ciclos y condicionales.

PRUEBAS

Una sesión sin nadie, un estudiante que asistió a todas, nombre repetido en la misma sesión y total de sesiones igual a cero.

7 · Bitácora de sensores

Un sensor registró su estado cada hora. La bitácora es una **tupla** porque un registro ya tomado no se debe poder alterar.

Entrada. Tupla de estados: ("OK", "OK", "FALLA", "OK", "ALERTA", ...).

Salida. conteo, primera_falla, posiciones_falla y bitacora_completa.

QUÉ DEBE HACER

1. Cuenta cuántas veces aparece cada estado distinto, **usando** `count()`.
2. Encuentra en qué hora ocurrió la **primera** "FALLA".
3. Encuentra **todas** las posiciones donde hay "FALLA". *Pista: `index()` acepta un segundo argumento que dice desde dónde buscar.*
4. Llega el turno de la noche como otra tupla. Únela a la bitácora en una **tupla nueva**, sin modificar la original.
5. Reporta qué pasa si pides la posición de un estado que **no está** en la bitácora.

EJEMPLO DE SALIDA

```
conteo = {"OK": 7, "FALLA": 3, "ALERTA": 2}
primera_falla = 2
posiciones_falla = [2, 6, 11]
```

RESTRICCIONES

La bitácora es inmutable: **no la conviertas a lista** para resolver los puntos 1 a 3.

PRUEBAS

Bitácora vacía, un estado que no aparece nunca, y una bitácora de un solo elemento.

8 · Agenda de la sala de cómputo

Los grupos piden bloques de una hora en la sala. La agenda debe respetar el límite diario y unir horas consecutivas del mismo grupo.

Entrada. Tuplas (grupo, día, hora) con hora entera entre 0 y 23.

Salida. agenda[(día, grupo)] : lista de bloques (inicio, fin, tipo) · rechazados[indice] : motivo.

QUÉ DEBE HACER

1. Procesa las solicitudes **en orden de llegada**. Acepta horas de inicio entre 6 y 20 inclusive; rechaza las demás con el motivo "fuera_de_horario".
2. Acepta máximo **dos solicitudes** por grupo y día. Cada solicitud cuenta como un turno, aunque después se una con otra. Rechaza los excedentes como "exceso_turnos".
3. Representa la hora `h` como el intervalo `[h, h+1)`. Dos horas consecutivas se combinan en un bloque de tipo "continuo"; las demás quedan como "simple".
4. Guarda en `rechazados` el **índice original** de la solicitud, empezando en cero.
5. Decide y declara qué haces con una hora duplicada del mismo grupo y día.

EJEMPLO DE SALIDA

```
# Grupo A pide las horas 9 y 10 del lunes
agenda = {"lunes", "A"): [(9, 11, "continuo")]}
rechazados = {3: "fuera_de_horario"}
```

RESTRICCIONES

Usa tuplas como claves. Funciones y ciclos; sin librerías externas.

PRUEBAS

Horas 5, 6, 20 y 21; dos solicitudes separadas; horas consecutivas pedidas en orden inverso; una tercera solicitud del mismo grupo.

9 · Producto con validación

El sistema de la cafetería necesita representar un producto cuyo precio **nunca** pueda quedar en un valor imposible.

Entrada. nombre, precio y cantidad.

Salida. Clase Producto.

QUÉ DEBE HACER

1. Guarda el precio en `__precio` y exponlo con una `@property` llamada `precio`.
2. El `setter` debe rechazar precios negativos lanzando `ValueError`.
3. `valor_total()` retorna `precio * cantidad`.
4. `__repr__` muestra el producto de forma legible.
5. `__eq__`: dos productos son el mismo si tienen **el mismo nombre**, sin importar precio ni cantidad.

VERIFICACIÓN MÍNIMA

```
p = Producto("Empanada", 2000, 30)
print(p.precio)           # 2000    <- sin parentesis: es una propiedad
p.precio = 2500           # pasa por el setter
p.precio = -100           # debe lanzar ValueError
print(Producto("Jugo", 3000, 5) == Producto("Jugo", 9999, 1)) # True
```

RESTRICCIONES

El precio **no** puede quedar accesible como atributo público.

PRUEBAS

Precio cero, precio negativo en el constructor y en el `setter`, y dos productos con el mismo nombre.

10 · Pedido como colección

Un pedido agrupa varios productos y debe comportarse como una colección de Python.

Entrada. Objetos `Producto` .

Salida. Clase `Pedido` .

QUÉ DEBE HACER

1. `agregar(producto)` y `retirar(nombre)` . Retirar algo que no está debe lanzar `ValueError` .
2. `total()` suma el `valor_total()` de todos los productos.
3. Implementa los protocolos para que funcionen: `len(pedido)` , `for p in pedido` , `"Empanada" in pedido` y `print(pedido)` .
4. La colección interna se declara con **doble guion bajo** y **no puede alcanzarse desde fuera**. Ningún método entrega la lista viva; si exposes el contenido, entrega una **tupla**.

VERIFICACIÓN MÍNIMA

```
pedido = Pedido()
pedido.agregar(Producto("Empanada", 2000, 3))
pedido.agregar(Producto("Jugo", 3000, 2))
```

```
print(len(pedido))           # 2
print("Jugo" in pedido)     # True
print(pedido.total())       # 12000
pedido.__datos              # debe fallar
```

RESTRICCIONES

Prohibido exponer la colección interna. `in` debe funcionar con el **nombre** del producto.

PRUEBAS

Pedido vacío, retirar un producto inexistente, y comprobar que `list(pedido)` entrega una copia.

11 · Pedido con descuento

La cafetería aplica descuentos a los pedidos de los empleados.

Entrada. Un porcentaje de descuento.

Salida. Clase `PedidoConDescuento`, que hereda de `Pedido`.

QUÉ DEBE HACER

1. El constructor recibe el porcentaje y llama a `super().__init__()`.
2. Rechaza porcentajes fuera del rango 0 a 100 con `ValueError`.
3. Sobrescribe `total()` aplicando el descuento, **reutilizando** el total del padre con `super()`. No vuelvas a recorrer los productos.
4. Sobrescribe `__repr__` para que se note que es un pedido con descuento.
5. Comprueba que `len()`, `in` y el recorrido **siguen funcionando** sin que los hayas vuelto a escribir.

VERIFICACIÓN MÍNIMA

```
p = PedidoConDescuento(10)
p.agregar(Producto("Empanada", 2000, 5))
print(p.total())           # 9000.0
print(len(p))              # 1      <- heredado
print(isinstance(p, Pedido)) # True
```

RESTRICCIONES

`total()` debe apoyarse en `super().total()`.

PRUEBAS

Descuento 0, descuento 100, descuento negativo y mayor que 100.

12 · Medios de pago

La caja acepta efectivo, tarjeta y transferencia. Cada uno cobra distinto, pero la caja no quiere preguntar de qué tipo es cada pago.

Entrada. Una lista con objetos de distintos tipos de pago.

Salida. Función `procesar_pagos(pagos, monto)`.

QUÉ DEBE HACER

1. Crea tres clases — `Efectivo`, `Tarjeta`, `Transferencia` — **sin superclase común**. Cada una implementa `cobrar(monto)` y `descripcion()`.
2. `Efectivo` no cobra recargo. `Tarjeta` suma un 3 %. `Transferencia` suma una comisión fija de 1500.
3. Escribe `procesar_pagos(pagos, monto)` que recorra la lista, llame a `cobrar` de cada uno e imprima la descripción con el valor final. **No puede usar `isinstance` ni preguntar el tipo.**
4. Agrega una cuarta clase `PuntosFidelidad` que descuente un 5 %, **sin tocar la función** del punto 3.

VERIFICACIÓN MÍNIMA

```
procesar_pagos([Efectivo(), Tarjeta(), Transferencia()], 10000)
# Efectivo      -> 10000
# Tarjeta       -> 10300.0
# Transferencia -> 11500
```

RESTRICCIONES

Prohibido `isinstance`, `type()` y cadenas de `if` sobre el tipo. Esto se resuelve con *duck typing*.

PRUEBAS

Lista vacía, monto cero, y agregar la cuarta clase comprobando que la función no cambió.

13 · Esquema de evaluación

El curso evalúa con talleres, parciales y un proyecto. Cada uno calcula su nota distinto, pero todos deben saber hacerlo.

Entrada. Peso de cada evaluación y sus datos propios.

Salida. Jerarquía Evaluacion con tres subclases.

QUÉ DEBE HACER

1. Crea una clase **abstracta** `Evaluacion` con el atributo `peso` y el método abstracto `nota()`.
2. Dale un método **concreto** `aporte()` que retorne `nota() * peso / 100`. Las subclases lo heredan tal cual: no lo reescriben.
3. Implementa `Taller` (puntos sobre un total), `Parcial` (respuestas correctas sobre el total) y `Proyecto` (promedio de tres rúbricas).
4. Escribe `definitiva(evaluaciones)` que sume los aportes.
5. Crea una clase `Quiz(Evaluacion)` **sin implementar** `nota()` e intenta instanciarla. Muestra el error y explica en un comentario qué lo provocó.

VERIFICACIÓN MÍNIMA

```
evs = [Taller(30, 38, 40), Parcial(70, 17, 20)]
for e in evs:
    print(type(e).__name__, e.nota(), e.aporte())
print("definitiva:", definitiva(evs))
```

RESTRICCIONES

Usa `abc.ABC` y `@abstractmethod`. Valida que los pesos sumen 100 y repórtalo si no.

PRUEBAS

Pesos que no suman 100, una evaluación con total cero, y la clase incompleta del punto 5.

14 · Navegador con historial

Simula un navegador con una página actual y **dos pilas**: las páginas anteriores y aquellas a las que todavía se puede avanzar.

Entrada. Comandos `visitar(url)`, `atras()`, `adelante()`, `actual()`.

Salida. Clase `Navegador`.

ESTRUCTURAS E INVARIANTES

Usa dos objetos `Pila`. Guarda la página actual en un atributo aparte; **nunca** la tengas al mismo tiempo dentro de una de las pilas. Los atributos no pueden quedar accesibles desde fuera.

TRANSICIONES REQUERIDAS

1. **Visitar:** si hay página actual, apíjala en la pila de atrás. La nueva URL pasa a ser la actual y **se invalida todo el historial de adelante**.
2. **Atrás:** apila la actual en la pila de adelante y desapila la anterior, que pasa a ser la actual. Si no hay a dónde ir, retorna `None` y no cambia nada.
3. **Adelante:** la transición simétrica.
4. **Actual:** retorna la página actual, o `None` si todavía no se ha visitado nada.

TRAZA MÍNIMA

```
visitar("uni.edu") → visitar("uni.edu/notas") → visitar("uni.edu/notas/2026") →
actual() → atras() → atras() → adelante() → visitar("uni.edu/horario") → adelante()
```

Debe dar: `atras()` retorna `uni.edu/notas` y luego `uni.edu`; `adelante()` retorna `uni.edu/notas`; el último `adelante()` retorna `None` porque visitar borró el camino.

PRUEBAS

Estado inicial sin visitas, `atras()` con la pila vacía, visitas consecutivas, y comprobar que tras visitar no queda nada por rehacer.

15 · Editor con deshacer y rehacer

Un editor de texto guarda cada fragmento escrito para poder deshacerlo.

Entrada. `escribir(texto)`, `deshacer()`, `rehacer()`, `contenido()`.

Salida. Clase `Editor`.

QUÉ DEBE HACER

1. `escribir(texto)` apila el fragmento en la pila de deshacer y **vacía** la pila de rehacer.
2. `deshacer()` mueve el último fragmento a la pila de rehacer y lo retorna. Si no hay nada, retorna `None`.
3. `rehacer()` hace lo simétrico.
4. `contenido()` arma el texto completo **en orden de escritura**. Ojo: una pila se recorre al revés.

TRAZA MÍNIMA

```
escribir("Hola") → escribir(" mundo") → escribir("!") → contenido() → deshacer() →  
deshacer() → contenido() → rehacer() → contenido() → escribir(" clase") → rehacer()
```

Debe dar: `"Hola mundo!"`, luego `"Hola"`, luego `"Hola mundo"`, y el último `rehacer()` retorna `None`.

RESTRICCIONES

Dos objetos `Pila`. Prohibido guardar el texto completo en una cadena aparte: debe reconstruirse.

PRUEBAS

Deshacer con el editor vacío, rehacer sin haber deshecho, y escribir después de deshacer.

16 · Fila de la cafetería

La cafetería atiende en orden de llegada, pero a veces alguien se va sin esperar.

Entrada. `llega(nombre)`, `atender()`, `se_retira(nombre)`, `estado()`.

Salida. Clase `FilaCafeteria`.

QUÉ DEBE HACER

1. `llega(nombre)` encola a la persona al final.
2. `atender()` desencola al del frente, lo retorna y lleva la cuenta de **cuántos se han atendido**.
3. `se_retira(nombre)` saca a esa persona **sin atenderla**, conservando el orden de los demás. *Pista: desencola todo y vuelve a encolar lo que no sea esa persona.*
4. `estado()` retorna a quién le toca, quién es el último y cuántos esperan.

TRAZA MÍNIMA

```
Llegan Ana , Luis , Sara , Pedro → estado() → atender() → atender() → llega Camila →  
se_retira("Sara") → estado()
```

Debe dar: al final esperan `Pedro` y `Camila`, en ese orden, con 2 personas atendidas.

RESTRICCIONES

Usa tu clase `Cola`. Prohibido convertirla a lista para resolver el punto 3.

PRUEBAS

Atender con la fila vacía, retirar a alguien que no está, y retirar al que está de primero.

17 · Notificaciones del celular

La pantalla del celular solo muestra las **últimas 3** notificaciones. Cuando llega una nueva y no hay espacio, la más vieja desaparece.

Entrada. `llega(app)`, `visibles()`.

Salida. Clase `Notificaciones`, construida con una `ColaCircular` de capacidad fija.

QUÉ DEBE HACER

1. El constructor recibe la **capacidad** y crea la cola circular.
2. `llega(app)` encola la notificación. Si ya está llena, **descarta la más vieja** y la retorna para poder informar cuál desapareció. Si había espacio, retorna `None`.
3. `visibles()` retorna las notificaciones en orden, de la más vieja a la más nueva.
4. La capacidad **no puede cambiar** después de crear el objeto.

TRAZA MÍNIMA

capacidad 3 → llegan "WhatsApp", "Correo", "Calendario" → `visibles()` → llega "Banco" → llega "Instagram" → `visibles()`

Debe dar: "Banco" desplaza a "WhatsApp", "Instagram" desplaza a "Correo", y al final se ven Calendario, Banco, Instagram.

RESTRICCIONES

El arreglo interno **no puede crecer**: tras 20 notificaciones debe seguir teniendo capacidad 3.

PRUEBAS

Capacidad 1, menos notificaciones que la capacidad, y exactamente la capacidad.

18 · Turnos rotativos del laboratorio

Tres equipos se turnan el microscopio. Cada uno necesita una cantidad distinta de turnos y, cuando termina uno, vuelve al final de la fila si aún le falta.

Entrada. Diccionario {"E1": 3, "E2": 1, "E3": 2} con los turnos que necesita cada equipo.
Salida. orden_de_uso y total_turnos .

QUÉ DEBE HACER

1. Encola los equipos en el orden dado.
2. Mientras quede alguno con turnos pendientes:
 - saca el del frente,
 - regístralo en orden_de_uso ,
 - réstale un turno,
 - si todavía le quedan, **vuélvelo a encolar al final**.
3. Retorna el orden completo de uso y cuántas veces se usó el microscopio en total.
4. Un equipo que pida **cero turnos** no debe aparecer nunca en orden_de_uso .

EJEMPLO DE SALIDA

```
orden_de_uso = ["E1", "E2", "E3", "E1", "E3", "E1"]  
total_turnos = 6
```

RESTRICCIONES

Usa Cola . Si tu clase tiene rotar() , decide si te sirve acá y justifica por qué sí o por qué no.

PRUEBAS

Un solo equipo, un equipo con cero turnos, y todos con un turno.

19 · Sala de urgencias

Una sala de urgencias atiende por **prioridad**, no por orden de llegada, y guarda el historial de lo que hizo para poder revertirlo.

Entrada. `llega(nombre, nivel)` con nivel 1 (crítico), 2 (urgente) o 3 (general).

Salida. Clase Urgencias con `atender()`, `deshacer_ultima()` y `estado()`.

QUÉ DEBE HACER

1. Mantén **tres colas**, una por nivel. `llega` valida el nivel y encola en la que corresponde.
2. `atender()` saca siempre del nivel más urgente **que no esté vacío**. Retorna el paciente y lo registra en una **pila** de atenciones.
3. `deshacer_ultima()` saca al último atendido de la pila y lo **devuelve al frente** de su cola. *Pista: tu Cola encola al final; piensa cómo lograr que quede de primero.*
4. `estado()` retorna cuántos esperan en cada nivel y quién fue el último atendido.
5. Si todas las colas están vacías, `atender()` retorna `None` sin fallar.

TRAZA MÍNIMA

Llegan Ana (3), Luis (1), Sara (2), Pedro (1) → `atender()` → `atender()` → `estado()` → `deshacer_ultima()` → `estado()`

Debe dar: se atiende primero Luis y luego Pedro (los dos de nivel 1). Tras deshacer, Pedro vuelve a estar de primero en la cola de nivel 1.

RESTRICCIONES

Tres objetos Cola y un objeto Pila. No uses una sola lista ordenada.

PRUEBAS

Nivel inválido, deshacer sin haber atendido, y atender con todas las colas vacías.

20 · Despacho de domicilios

Un restaurante recibe pedidos, los prepara en orden y despacha con una flota de repartidores que rotan. Integra todo lo del taller.

Entrada. Pedidos con productos (ejercicios 9 y 10) y una lista de repartidores.

Salida. Clase `Despacho`.

QUÉ DEBE HACER

1. `recibir(pedido)` encola el pedido en la **cola de preparación**.
2. `preparar()` saca el siguiente pedido y lo pasa a la **cola de listos**.
3. `despachar()` saca un pedido de listos y lo asigna al **siguiente repartidor** de una cola circular de repartidores, que **rota**: después de entregar vuelve al final.
4. Lleva un diccionario `entregas_por_repartidor` con cuántos pedidos llevó cada uno.
5. `resumen()` retorna cuántos pedidos hay en cada etapa, el total facturado y el repartidor con más entregas.
6. Si un pedido tiene descuento (ejercicio 11), el total facturado debe reflejarlo **sin que** `Despacho` **pregunte de qué tipo es**.

VERIFICACIÓN MÍNIMA

```
d = Despacho(["Carlos", "Marta"])
d.recibir(pedido_a); d.recibir(pedido_b); d.recibir(pedido_c)
d.preparar(); d.preparar(); d.despachar()
print(d.resumen())
```

RESTRICCIONES

Reutiliza `Producto`, `Pedido` y `PedidoConDescuento`. El punto 6 se resuelve con polimorfismo: nada de `isinstance`.

PRUEBAS

Despachar sin pedidos listos, un solo repartidor, y un pedido vacío.

DISEÑAR · IMPLEMENTAR · VERIFICAR

Evaluación

NOTA

Parcial 1 — 30 %. Se presenta en **papel**, en clase. Cubre los mismos temas de este taller: ciclos, listas, tuplas, diccionarios, funciones, objetos y estructuras lineales.

(Pendiente: fecha y condiciones de presentación.)